

DIGITAL LOGIC

ENEX 152 — Master Study & Revision Guide

A printable, exam-oriented guide covering the complete syllabus from digital fundamentals to sequential machine design and digital ICs.

Course	Digital Logic
Code	ENEX 152
Lecture / Tutorial / Practical	3 / 1 / 3
Year / Part	I / II
Syllabus units	8
Study method	Concept → Truth Table → Equation → Design → Practice

How to use this book: First read the concept pages without memorizing. Then reproduce truth tables and circuit/design procedures from memory. Finally solve the practice checklist at the end. Digital logic punishes passive reading with remarkable efficiency.

How to Master This Course

This guide is deliberately organized around the dependencies between topics. You should not treat the syllabus as eight unrelated chapters.

- 1 **Foundation:** number systems, codes, complements, signed numbers and logic levels.
- 2 **Logic algebra:** gates, Boolean laws, truth tables, SOP/POS and K-maps.
- 3 **Combinational design:** adders, subtractors, MUX/DEMUX, encoders, decoders, comparators and displays.
- 4 **Sequential logic:** latches, flip-flops, characteristic/excitation tables, timing and conversions.
- 5 **Storage/counting:** registers and asynchronous/synchronous counters.
- 6 **Machine design:** state diagrams, state reduction/assignment, excitation maps and realization.
- 7 **Hardware logic families:** TTL, CMOS, switching characteristics and applications.

Core symbols

Symbol	Meaning
A'	NOT A / complement of A
AB	A AND B
$A + B$	A OR B
$\oplus$	XOR
$\boxplus$	XNOR
$\Sigma m(\dots)$	Sum of minterms
$\Pi M(\dots)$	Product of maxterms
X	Don't-care condition

1. Introduction

1.1 Digital vs Analog Signals

Analog	Digital
Continuous amplitude/time	Discrete levels
More sensitive to noise accumulation	Can be regenerated/restored
Examples: microphone voltage, temperature sensor	Computer data, logic circuits, digital communication

Key idea: Digital circuits usually interpret a voltage range as logic 0 or logic 1. The exact voltage ranges depend on the logic family and device.

1.2 Logic-Level Diagram

A logic-level diagram shows the voltage ranges assigned to logic states. Real circuits do not require one exact voltage for every state; they specify allowable ranges and noise margins.

1.3 Digital Integrated Circuits

An IC integrates many electronic components on one semiconductor chip. Digital ICs include gates, flip-flops, counters, memories, processors and interface circuits.

1.4 Clock Triggering

Sequential circuits use a clock to coordinate state changes. A flip-flop may respond to a rising edge, falling edge, or level depending on its design.

Rising edge: $0 \rightarrow 1$ Falling edge: $1 \rightarrow 0$

1.5 Digital-System Applications

- Computers and microcontrollers
- Communication systems
- Industrial control
- Consumer electronics
- Instrumentation and measurement
- Automotive and embedded systems

1.6 Number Systems

System	Base	Digits
Decimal	10	0–9
Binary	2	0, 1
Octal	8	0–7
Hexadecimal	16	0–9, A–F

Conversion rules: Binary $\rightarrow$ octal: group bits in 3s from the radix point. Binary $\rightarrow$ hexadecimal: group bits in 4s. Decimal $\rightarrow$ binary: repeated division by 2 for the integer part; repeated multiplication by 2 for a fractional part.

Decimal 45 $\rightarrow$ binary: $45 = 32 + 8 + 4 + 1 = (101101)_{\text{■}}$

Binary $101101_{\text{■}}$ $\rightarrow$ octal: $101\ 101 \rightarrow 55_{\text{■}}$ Binary $101101_{\text{■}}$ $\rightarrow$ hexadecimal: $0010\ 1101 \rightarrow 2D_{\text{■}}$

1.6.2 BCD

BCD represents each decimal digit separately using four bits. It is not the same as ordinary binary representation.

$59_{\text{■}}$ $\rightarrow$ BCD = $0101\ 1001\ 59_{\text{■}}$ $\rightarrow$ binary = $111011_{\text{■}}$

In 8421 BCD, only 0000 through 1001 are valid digit codes.

1.6.3 Excess-3

Add 3 to each decimal digit, then encode the result in 4-bit binary.

$59 \rightarrow$ digits 5 and 9 $\rightarrow 5+3=8 \rightarrow 1000$; $9+3=12 \rightarrow 1100$ Therefore Excess-3(59) = $1000\ 1100$

1.6.4 Gray Code

Gray code changes only one bit between adjacent code words. A common binary-to-Gray conversion is:

$$G_{\text{■}} = B_{\text{■}} \quad G_{\text{■}} = B_{\text{■}} \oplus B_{\text{■}}$$

Example: Binary $1011 \rightarrow$ Gray 1110 .

1.7 ASCII and EBCDIC

ASCII is a character encoding widely used in computing and communication. EBCDIC is another character encoding historically associated with IBM systems. For exams, know their purpose, broad structure and typical applications rather than treating them as number systems.

1.8 1's and 2's Complement

1's complement: invert every bit. 2's complement: 1's complement + 1.

8-bit +5 = 00000101 1's complement = 11111010 2's complement = 11111011

Two's complement is widely used for signed integer arithmetic because addition/subtraction can use the same binary adder hardware.

1.9 Signed Number Representation

Method	Main idea
Sign-magnitude	MSB is sign; remaining bits give magnitude
1's complement	Negative value obtained by bit inversion
2's complement	Negative value obtained by inversion + 1

For n-bit two's-complement numbers, the range is -2^{n-1} to $+2^{n-1}-1$.

2. Logic Gates

2.1 Basic Gates

A	B	AND AB	OR A+B	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT: $0 \rightarrow 1$ and $1 \rightarrow 0$. **AND** is 1 only when all inputs are 1. **OR** is 1 when at least one input is 1. **XOR** is 1 when inputs differ. **XNOR** is 1 when inputs are equal.

2.2 Universal Gates

NAND and NOR are universal gates because any Boolean function can be constructed using only NAND gates or only NOR gates.

Using NAND: NOT $A = A \text{ NAND } A$ AND: $AB = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$ Using NOR: NOT $A = A \text{ NOR } A$ OR: $A+B = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$

2.3 Exclusive Gates

XOR detects inequality; XNOR detects equality.

$$A \oplus B = A'B + AB' \quad A \boxplus B = AB + A'B'$$

2.4 Positive and Negative Logic

The same physical voltage levels can be interpreted differently depending on whether the lower or higher level represents the asserted logic state.

2.5 De Morgan's Laws

$$(AB)' = A' + B' \quad (A+B)' = A'B'$$

A useful memory trick: when the complement crosses a bracket, AND becomes OR and OR becomes AND, while each variable is complemented.

2.6 Applications

- AND: enable/interlock conditions
- OR: multiple request sources
- NOT: inversion
- XOR: parity generation/checking and addition
- XNOR: equality detection
- NAND/NOR: general-purpose logic implementation

3. Boolean Algebra and K-Maps

3.1 Essential Boolean Laws

Law	Expression
Identity	$A+0=A$; $A \cdot 1=A$
Null	$A+1=1$; $A \cdot 0=0$
Idempotent	$A+A=A$; $AA=A$
Complement	$A+A'=1$; $AA'=0$
Involution	$(A')'=A$
Absorption	$A+AB=A$; $A(A+B)=A$
Commutative	$A+B=B+A$; $AB=BA$
Associative	$A+(B+C)=(A+B)+C$; $A(BC)=(AB)C$
Distributive	$A(B+C)=AB+AC$; $A+BC=(A+B)(A+C)$

3.2 Simplification

Example: $F = A + AB$ $F = A(1+B) = A$

Example: $F = AB + AB'$ $F = A(B+B') = A$

The goal is to reduce gate count, gate inputs, propagation delay, power or hardware complexity while preserving the function.

3.3 Minterms and Maxterms

A minterm is a product term containing every variable exactly once. A maxterm is a sum term containing every variable exactly once.

For a truth table, Σm lists rows where $F=1$. ΠM lists rows where $F=0$.

3.4 SOP and POS

SOP: OR of AND terms. **POS:** AND of OR terms.

SOP: $F = A'B + AB'$ POS: $F = (A+B)(A'+B')$

3.5–3.9 K-Maps

Karnaugh maps are a visual simplification method. Adjacent cells differ in exactly one variable. Group 1, 2, 4, 8, ... cells. Groups may wrap around edges.

- **Pairs, quads, octets:** use the largest valid groups possible.
- **Overlap:** a cell may belong to multiple groups if that produces a simpler expression.
- **Redundant groups:** omit a group if all its cells are already covered without losing required minterms.
- **Don't-care X:** may be treated as 0 or 1, whichever gives a simpler result.
- **Rolling/wrap-around:** left-right and top-bottom edges can be adjacent.

Four-variable map order: use Gray-code ordering, typically 00, 01, 11, 10 along each axis.

Example: $F(A,B,C,D)=\Sigma m(0,2,8,10)$ The four cells form a quad. B and D change; $A=0/1$ and $C=0$, so $F = C'D'$.

For every K-map question, write: (1) map, (2) groups, (3) simplified terms, (4) final expression. Do not jump directly to the answer.

4. Combinational Logic Circuits

A combinational circuit's output depends only on its present inputs. There is no stored state.

4.1 Design Procedure

- 1 Define inputs and outputs.
- 2 Construct the truth table.
- 3 Write Boolean expressions.
- 4 Simplify using Boolean algebra or K-map.
- 5 Implement with gates or required building blocks.
- 6 Verify the circuit against the truth table.

4.2 Half Adder

A	B	Sum S	Carry C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A \oplus B \quad C = AB$$

4.2 Full Adder

A full adder adds A, B and carry-in C_{in} .

$$S = A \oplus B \oplus C_{in} \quad C_{out} = AB + C_{in}(A \oplus B) \quad \text{Equivalent: } C_{out} = AB + AC_{in} + BC_{in}$$

4.3 Half Subtractor

$$\text{Difference } D = A \oplus B \quad \text{Borrow } B_{out} = A'B$$

4.3 Full Subtractor

$$D = A \oplus B \oplus B_{in} \quad B_{out} = A'B + A'B_{in} + BB_{in}$$

4.4 Ripple Carry and Fast Adders

A ripple-carry adder chains full adders. The carry must propagate through stages, creating delay. Carry-lookahead and other fast-adder approaches reduce carry propagation delay by generating carry information more quickly.

4.5 Multiplexer

A MUX selects one of many inputs and sends it to one output. An n-to-1 MUX needs $\log_2(n)$ select lines.

$$2:1 \text{ MUX: } Y = S'I_0 + SI_1$$

MUXes can also implement Boolean functions by connecting data inputs to 0, 1 or variables.

4.6 Demultiplexer

A DEMUX takes one input and routes it to one of several outputs according to select lines.

4.7–4.9 Encoders

An encoder converts one active input among many into a binary code. A priority encoder resolves cases where multiple inputs are active by assigning priority.

4.10–4.12 Decoders and Displays

A decoder converts an n-bit input code into one of up to 2^n active outputs. A BCD-to-decimal decoder maps decimal digits to ten output lines. A seven-segment decoder drives the segments a–g to display decimal digits.

4.13 Magnitude Comparator

For two 1-bit inputs: $A > B = AB'$ $A < B = A'B$ $A = B = A \text{ XNOR } B$

Multi-bit comparators compare from the most significant bit downward. The first unequal bit determines the result.

5. Sequential Logic Circuits

A sequential circuit depends on present inputs *and stored state*. This is the central distinction from combinational logic.

5.1 Latches and Flip-Flops

Device	Core behavior
SR	Set/reset storage; invalid condition depends on implementation
D	Next state follows D at the active clock event
T	T=0 holds; T=1 toggles
JK	J=1,K=0 set; J=0,K=1 reset; J=K=1 toggle

5.2 Characteristic Equations

D FF: $Q^{\bullet} = D$ T FF: $Q^{\bullet} = T \oplus Q$ JK FF: $Q^{\bullet} = JQ' + K'Q$ SR FF (common active-high form): $Q^{\bullet} = S + R'Q$, with S=R=1 prohibited

Excitation Tables

Present Q	Next Q [■]	D	T	J	K
0	0	0	0	0	X
0	1	1	1	1	X
1	0	0	1	X	1
1	1	1	0	X	0

X means don't care. Excitation tables are used when designing a circuit to produce required state transitions.

5.3 Master-Slave Flip-Flop

A master-slave arrangement uses two stages controlled on opposite clock phases so the output changes in a controlled manner and helps avoid certain race-through problems.

5.4 Timing Diagrams

When reading timing diagrams, identify the active clock edge first. Then apply the flip-flop's characteristic behavior. Mark Q changes only at the relevant triggering event.

5.5–5.7 Applications

- Registers
- Counters
- Frequency division
- State machines
- Data storage
- Synchronization and control

5.6 Flip-Flop Conversion

General conversion method:

- 1 Choose the available flip-flop and desired next-state behavior.

- 2 Construct a present-state/desired-next-state table.
- 3 Use the excitation table of the available flip-flop.
- 4 Find required input expressions with K-map.
- 5 Implement and verify.

6. Registers and Counters

6.1 Register Fundamentals

A register is a group of flip-flops used to store a multi-bit word. Register type describes how data enters and/or leaves the register.

6.2 SISO, SIPO, PISO, PIPO

Type	Input	Output	Typical use
SISO	Serial	Serial	Delay/data shifting
SIPO	Serial	Parallel	Serial-to-parallel conversion
PISO	Parallel	Serial	Parallel-to-serial conversion
PIPO	Parallel	Parallel	Temporary storage

6.3 Data Transfer Timing

Track one bit per clock edge. A shift register moves stored data one stage per active clock event, depending on the architecture.

6.4 Asynchronous Counters

In an asynchronous/ripple counter, only the first flip-flop receives the external clock directly. Later stages are triggered by preceding stages, causing cumulative propagation delay.

6.5 Up, Down and MOD-N

For a binary up-counter, states progress $000 \rightarrow 001 \rightarrow 010 \rightarrow \dots \rightarrow 111 \rightarrow 000$.

A MOD-N counter has N distinct states. If N is not a power of two, unused states must be handled/reset appropriately.

6.6–6.7 Synchronous Counters

All flip-flops receive the same clock in a synchronous counter. Additional logic determines which stages toggle. This reduces ripple delay and makes timing more predictable.

Design idea: determine the required state sequence $\rightarrow$ construct transition table $\rightarrow$ derive flip-flop inputs using excitation tables $\rightarrow$ simplify $\rightarrow$ draw circuit.

6.8 Applications

- Event counting
- Frequency division
- Digital clocks
- Timers
- Sequence generation
- Address generation

7. Sequential Machine Designs

This unit combines everything before it. If Units 3–6 are shaky, this chapter becomes a small bureaucratic nightmare of tables. Fix the foundations first.

7.1 Machine Design Procedure

- 1 State the specification and identify inputs/outputs.
- 2 Construct the state diagram.
- 3 Create the transition/flow table.
- 4 Reduce equivalent or redundant states when appropriate.
- 5 Assign binary codes to states.
- 6 Choose flip-flop type.
- 7 Construct excitation requirements.
- 8 Simplify input equations with K-maps.
- 9 Draw and verify the final synchronous machine.

7.2 Primitive State Diagrams

A primitive state diagram represents the minimum state behavior before state reduction/assignment. Arrows represent transitions and labels show input/output information depending on the machine model.

7.3 Transition / Flow Tables

A transition table maps present state and input to next state and, for output-producing machines, the output.

7.4 Redundant States

Two states may be equivalent if they produce the same required outputs and transition in equivalent ways for every relevant input. State reduction can decrease hardware.

7.5 Binary Assignment

Assign a binary code to each remaining state. With N states, at least $\lceil \log_2 N \rceil$ flip-flops are required.

7.6 Excitation Maps

After state assignment, determine the required flip-flop input for every transition. Use K-maps to simplify each input equation.

7.7–7.8 Realization

Realization means converting the abstract state machine into actual flip-flops and combinational logic. The final circuit should show state-storage elements, next-state logic, inputs and outputs.

7.9 Sequence Detectors

A sequence detector recognizes a specified bit pattern. The state diagram represents how much of the target sequence has been matched so far.

One-bit/two-bit input sequence detectors: carefully define whether overlapping sequences are allowed. Overlap changes the state transitions.

Typical workflow: target sequence $\rightarrow$ states representing matched prefix $\rightarrow$ transitions for 0/1 $\rightarrow$ output condition $\rightarrow$ flip-flop implementation

8. Digital Integrated Circuits

8.1 BJT and MOSFET Switching

Digital circuits use transistors as switches. A BJT can be driven between cutoff and saturation in switching applications. MOSFETs are voltage-controlled devices and are fundamental to CMOS logic.

8.2 TTL Parameters

TTL uses bipolar transistors. Important parameters include propagation delay, power dissipation, fan-out, noise margin, supply voltage and switching characteristics.

8.3 TTL NAND, NOT, NOR

Know the basic transistor-level idea and logic behavior of TTL implementations, plus how the NAND/NOT/NOR functions are realized and interpreted.

8.4 CMOS Parameters

CMOS uses complementary MOSFET networks. Important parameters include propagation delay, power dissipation, noise margin, fan-out and supply voltage. CMOS has very high input impedance, while dynamic power is strongly related to switching activity.

8.5 CMOS NAND, NOR, NOT

CMOS logic is built from complementary pull-up and pull-down transistor networks. NAND and NOR differ in the series/parallel arrangement of the transistor networks.

8.6 Three-State Devices

A three-state output can produce logic 0, logic 1, or high impedance (Z). High impedance effectively disconnects the output from the shared bus, allowing controlled bus sharing.

8.7 Applications

- Multiplexed displays
- Frequency counters
- Time measurement
- Digital instrumentation

Master Tables to Memorize

Logic Gate Quick Sheet

Gate	Expression	Output = 1 when...
NOT	$Y=A'$	$A=0$
AND	$Y=AB$	all inputs are 1
OR	$Y=A+B$	at least one input is 1
NAND	$Y=(AB)'$	AND condition is false
NOR	$Y=(A+B)'$	all inputs are 0
XOR	$Y=A\oplus B$	inputs differ
XNOR	$Y=A\odot B$	inputs are equal

Flip-Flop Quick Sheet

FF	Characteristic equation	Key behavior
D	$Q_{\text{next}}=D$	copies D
T	$Q_{\text{next}}=T\oplus Q$	toggle when $T=1$
JK	$Q_{\text{next}}=JQ'+K'Q$	toggle when $J=K=1$
SR	$Q_{\text{next}}=S+R'Q$	set/reset; avoid invalid condition

Combinational vs Sequential

Feature	Combinational	Sequential
Depends on	Present inputs	Present inputs + state
Memory	No	Yes
Clock	Not required	Commonly required
Examples	Adder, MUX, decoder	Counter, register, FSM

Exam Problem Templates

Template A — Number Conversion

- 1 Write the source number and base.
- 2 Apply the correct conversion algorithm.
- 3 Show intermediate groups/divisions.
- 4 Write the final answer with its base.

Template B — Boolean/K-Map

- 1 Write the truth table or minterm list.
- 2 Draw the K-map in Gray-code order.
- 3 Place 1s and Xs.
- 4 Make the largest valid groups.
- 5 Write one simplified term per group.
- 6 Combine and verify.

Template C — Combinational Circuit

- 1 Identify inputs and outputs.
- 2 Truth table.
- 3 Boolean equations.
- 4 K-map simplification.
- 5 Gate/block implementation.
- 6 Verify at least representative cases.

Template D — Counter

- 1 Write required state sequence.
- 2 Determine number of flip-flops.
- 3 Create present/next-state table.
- 4 Use excitation table.
- 5 K-map each flip-flop input.
- 6 Draw common clock and logic.
- 7 Check unused states.

Template E — FSM / Sequence Detector

- 1 Define states in plain language first.
- 2 Draw transitions for every input.
- 3 Label outputs consistently.
- 4 Reduce states if possible.
- 5 Assign binary codes.
- 6 Derive excitation inputs.
- 7 Simplify and realize.
- 8 Test the target sequence and a near-miss sequence.

Common Traps

- **BCD $\neq$ binary:** encode each decimal digit separately in BCD.
- **Gray code ordering matters:** adjacent K-map cells must differ by one bit.
- **Largest K-map groups:** do not make tiny groups when a larger valid group exists.
- **Wrap-around is real:** edge cells can be adjacent. Humans invented maps and then forgot their edges connect.
- **XOR $\neq$ OR:** XOR is 1 only when inputs differ.
- **Ripple vs synchronous:** ripple counters do not clock every flip-flop simultaneously.
- **Characteristic vs excitation:** characteristic tells what the FF does; excitation tells what inputs are needed to obtain a desired transition.
- **Combinational vs sequential:** stored state is the dividing line.
- **MOD-N:** count states, not merely the number of flip-flops.
- **Timing diagrams:** identify triggering edge before changing Q.

Mastery Checklist

Do not mark a topic mastered because it looked familiar while reading. Mark it mastered only when you can reproduce the result without the book.

■	Skill
■	Convert decimal $\leftrightarrow$ binary $\leftrightarrow$ octal $\leftrightarrow$ hexadecimal.
■	Convert between binary, BCD, Excess-3 and Gray code.
■	Perform 1's and 2's complement and signed arithmetic.
■	Draw and explain every basic/universal/exclusive gate.
■	Apply De Morgan's laws without looking them up.
■	Simplify Boolean expressions using algebra.
■	Convert between truth tables, minterms/maxterms, SOP and POS.
■	Solve 2-, 3- and 4-variable K-maps including don't-cares and wrap-around.
■	Design a half/full adder.
■	Design a half/full subtractor.
■	Explain ripple-carry vs fast adders.
■	Implement a Boolean function using a MUX.
■	Design/understand DEMUX, encoder, priority encoder and decoder.
■	Explain BCD-to-decimal and seven-segment decoding.

■	Skill
■	Design and analyze a magnitude comparator.
■	Draw SR, D, T and JK characteristic/excitation tables.
■	Read flip-flop timing diagrams.
■	Perform flip-flop conversion.
■	Design SISO, SIPO, PISO and PIPO registers.
■	Analyze asynchronous and synchronous counters.
■	Design MOD-N, up and down counters.
■	Build a state diagram and transition table.
■	Reduce redundant/equivalent states.
■	Assign binary states and derive excitation equations.
■	Design a one-bit/two-bit sequence detector.
■	Compare TTL and CMOS parameters.
■	Explain three-state logic.
■	Explain multiplexed display, frequency-counter and time-measurement applications.

A Practical Revision Order

Phase	Topics	Goal
1	Unit 1	Number systems + codes + complements
2	Units 2–3	Gates + Boolean algebra + K-maps
3	Unit 4	Combinational circuit design
4	Unit 5	Flip-flops + timing + conversions
5	Unit 6	Registers + counters
6	Unit 7	FSM/state-machine design
7	Unit 8	TTL/CMOS + applications
8	Final	Mixed problems under time pressure

Suggested Study Loop

- 1 Read the concept for 20–30 minutes.
- 2 Close the guide and redraw the key table/diagram from memory.
- 3 Solve 3–5 problems without notes.
- 4 Check errors and write the reason for each error.
- 5 Repeat the missed problem the next day.
- 6 At the end of each unit, solve mixed questions rather than only repeating examples.

Final One-Page Mental Model

Digital Logic is one connected story:

Numbers/Codes → Boolean Logic → Gates → K-Maps → Combinational Circuits → Flip-Flops → Registers/Counters → State Machines → Digital IC Hardware

If you understand that chain, the syllabus stops being eight chapters and becomes one system. Number representations tell you how information is encoded. Boolean algebra describes logic. Gates implement the logic. Combinational circuits

calculate. Flip-flops store state. Registers store and move data. Counters and state machines control sequences. TTL/CMOS explain the physical implementation.

Mastery standard: You should be able to move in both directions: specification → truth table → equation → circuit, and circuit → behavior → truth table → equation. That two-way ability is what separates understanding from memorization.